How one file lands in the DB — followed as a story

> One example file, `13511_13571_06302026.csv` (patient 13511 · doctor 13571), travels **transcript → NLP → AI → screen**, and at each step we follow **which drawer (table) gets what**. Real stored values are shown too. Example sentence: **(utterance 66, sentence 4)** = "now, it's a whole lot lesser…"
>
> English mirror of `DB_TABLES_ROLES_KR.md`; if they diverge, the schema in `models.py` wins.

First — "table = drawer"

The DB is **a set of labeled drawers**, each holding a different kind of information. Processing one file spreads its data **across several drawers**. - The drawers the pipeline creates are all tied together by one `analysis_id` **(ticket number)**. (This file's ticket = **1**.)

Act 1 — the transcript arrives (de-identified)

The raw `SID22_doc2_06302026.xlsx` (real consultation transcript) is not kept as-is: patient/doctor names are **hashed** into `13511_13571_06302026.csv`. → **Nothing in the DB yet.** Just file prep.

Act 2 — create the analysis receipt → drawer ① `transcript_analysis_log`

When the pipeline starts on this file it makes **one "analysis receipt"** — the first drawer's first row. Its **id (=1)** becomes the `analysis_id` — the ticket stamped on every drawer made afterward.

> 🗄️ stored: `id=1`, `patient_id=13511`, `doctor_id=13571`, `total_sentences=438`, `top_n=10`, `ai_overall_score=0.8` (filled later), `processed=true` (when done)

Act 3 — NLP: split sentences and score them

3-1. split into sentences

Each utterance (one turn) is split into sentences, each getting an **address (utterance i, in-utterance i2)**. E.g. utterance 66 → 4 sentences, the 4th being **(66,4)** "now, it's a whole lot lesser…"

3-2. five graders score each sentence → drawer ② `nlp_all_predictions`

For every sentence, **5 models** (cp cancer-prognosis · le life-expectancy · ed erectile · inc incontinence · ius irritative-symptoms) score "probability this sentence is that topic". One row per sentence.

> 🗄️ (66,4): `pred_cp=0.781`, `pred_le=0.713`, `pred_ed=0.318`, `pred_inc=0.413`, `pred_ius=0.376` → both cp & le high = the sentence **spans two topics**. (All 438 sentences go in this drawer.)

3-3. keep only each topic's top-10 + context → drawer ③ `sentence_prediction`

Each model keeps its **top-10 sentences**, with surrounding context; the focus sentence is marked `<main>…</main>`. - (66,4) was picked into **both** cp and le top-10 → **2 rows** (one model=cp, one model=le).

> 🗄️ (66,4) cp row: `model=cp`, `pred_score=0.781`, `context="…85 to 90 percent…<main>now it's a whole lot lesser…</main>…"`

> (Side drawer `nlp_pipeline_intermediate` = a "working-notes" drawer of step snapshots — for debugging.)

Act 4 — AI (GPT-4o): summarize and score

4-1. score · extract · filter → drawer ④ `llm_pipeline_intermediate`

For each domain's top-10 candidates GPT-4o records: a 0-5 score + extracted estimate/treatment + whether the candidate survives (`survived_filter`). One row per candidate.

🔬 a surviving cp candidate: `domain=cp`, `ai_score=2`, `estimate=<missing>`, `survived_filter=true`, `score_explanation="…cancer mortality/survival or metastasis risk…"`

4-2. pick a domain representative and rewrite for the patient → drawer ⑤ `llm_domain_scoring_and_summary`

Each domain picks **one representative** and rewrites its sentence into **patient-friendly text**. **This drawer is the final result shown on the patient/doctor screens.**

🔬 cp: `ai_score=2`, `source_sentence="and they're essentially looking to ensure…"`, `reformat_sentence="You and your doctor did not discuss this information."` (patient-facing)

Finally the all-domain average (**0.8**) is written to receipt ①'s `ai_overall_score`, and `processed=true` is stamped.

Act 5 — make the patient-side parent key → drawer ⑥ `patient_summary`

A **parent key** (`patient_summary`: file, speaker) is created for patient-side data to reference. Later patient survey submissions reference it by FK (integrity · cascade · re-processing survival).

🔬 `patient_summary` = (`13511_13571_06302026.csv`, `Patient_13511_13571_06302026`)

Act 6 — the doctor opens the screen — which drawers feed it

The doctor dashboard **reads the drawers via the backend API**: - **Sentence grid** = sentences from ③ `sentence_prediction`, scores from ⑤ `llm_domain…`, side by side. - **Score summary / trajectory** = per-domain `ai_score` from ⑤. - When the doctor **rewrites a sentence** → new drawer ⑧ `doctor_rewrite_log` gets one row (original, revised, time) per rewrite.

Act 7 — the patient opens the screen — which drawers feed it

- Report **summary sentence** = ⑤ `llm_domain…`'s **`reformat_sentence`** (patient-facing rewrite).
- First-visit **Risk Perception answers** (sliders/choices) → drawer ⑦ `patient_survey_submission_log` (`survey_type='risk_perception_2'`, + REDCap `post_risk_perception_2`).
- **Follow-up surveys** (SDM/DCS/satisfaction) → the same drawer ⑦ `patient_survey_submission_log` (+ REDCap if enabled).

Act 8 — drawers that quietly fill in the background (behavior · recording)

Every **click/navigation** logs one behavior row: - Patient first-visit **report page** behavior → `patient_report_page_behavior` (report-only) - Patient **survey page** behavior (follow-up SDM/DCS/satisfaction + first-visit Risk survey) → `patient_followup_survey_page_behavior` - Doctor dashboard behavior → `doctor_behavior` - Screen recording (replay) → `session_recording` → These are **research usability logs** (behavior/timing only) — separate from answer values (values live in ⑦ `patient_survey_submission_log`).

Act 9 — the gatekeeper drawers (auth · access)

Drawers that guard who may see what: - `auth_user` (accounts) · `auth_api_key` (API keys). - (Plus `alembic_version` = a system drawer tracking the DB schema version.)

Appendix — step-by-step code map (file : function)

Step	File	Function
NLP A1 preprocess	`sentence_classification/preprocessing.py`	`identify_doctor_speaker` · `filter_doctor_rows`
NLP A2 segment	`sentence_classification/segmentation.py`	`segment_sentences`
NLP A3 5-model classify	`sentence_classification/classification.py`	`classify_all_models`
NLP A4 top-N select	`sentence_classification/selection.py`	`select_top_k_sentences`

Step	File	Function
NLP A5 attach context	`sentence_classification/context.py`	`add_context_column`
AI B1 score	`ai_pipeline/scoring.py`	`run_scoring`
AI B2 extract	`ai_pipeline/extraction.py`	`run_extraction`
AI B3 filter	`ai_pipeline/filtering.py`	`filter_candidates`
AI B4 select representative	`ai_pipeline/selection.py`	`run_selection`
AI B5 reformat	`ai_pipeline/reformat.py`	`run_reformat`
AI orchestration	`ai_pipeline/pipeline.py`	`run_domain_pipeline` / `run_ai_pipeline`
DB persist	`db/persistence_helper.py`	`persist_pipeline_results` → `_save_nlp_results` / `_save_ai_results`
DB persist (NLP tables)	`app/Backend/persistence.py`	`save_all`

Epilogue — the drawer map on one page

```
[receipt] transcript_analysis_log  (ticket analysis_id=1)
   │  this ticket ties the pipeline drawers below together
   ├─ ② nlp_all_predictions            5-model probs per sentence (all)
   ├─ ③ sentence_prediction            per-topic top-10 sentences + context (shared sentence = several rows)
   ├─   nlp_pipeline_intermediate      NLP working notes
   ├─ ④ llm_pipeline_intermediate      AI candidate score·extract·filter
   └─ ⑤ llm_domain_scoring_and_summary AI representative + patient rewrite  ← what the screen shows

[patient key] patient_summary (file, speaker)   ← parent anchor for patient survey submissions

[what the screens use]
   doctor: reads ③+⑤, on rewrite → ⑧ doctor_rewrite_log
   patient: reads ⑤'s reformat, on survey input → ⑦ patient_survey_submission_log
            (first-visit Risk = risk_perception_2, follow-up = sdm/dcs/satisfaction)

[in the background] behavior: patient_report_page_behavior·patient_followup_survey_page_behavior·doctor_behavior /
recording: session_recording
[gatekeepers] auth_user·auth_api_key
```

One-sentence summary: a file comes in → the **receipt (①)** makes a ticket → **NLP fills sentences/probs (②③)**, **AI fills scores/patient summaries (④⑤)** → the **screen reads ③⑤** for doctor/patient, while **what people entered/edited (survey ⑦ `patient_survey_submission_log` · doctor rewrites ⑧)** and **behavior logs** pile up separately.